

4)

```
import pandas as pd
import io

df =
pd.read_excel(io.BytesIO(uploaded[ '
Lab Session Data.xlsx'])))
```

```
numeric_df =
df.select_dtypes(include='number')
```

```
A = numeric_df.iloc[0].values
B = numeric_df.iloc[1].values
```

```
def minkowski_distance(A, B, p):

    total = 0

    for i in range(len(A)):
        total += abs(A[i] - B[i])**p

    return total ** (1/p)
```

```
print("Manhattan Distance =",
minkowski_distance(A, B, 1))
(A,B,1))
print("Euclidean Distance =",
minkowski_distance(A, B, 2))
minkowski_distance(A,B,2))
```

Output: Manhattan Distance = nan
Euclidean Distance = nan

5)

```
import pandas as pd
import matplotlib.pyplot as plt
import io
```

```
df =
pd.read_excel(io.BytesIO(uploaded[ '
Lab Session Data.xlsx'])))
```

```
numeric_df =
df.select_dtypes(include='number')
```

```
A = numeric_df.iloc[0].values
B = numeric_df.iloc[1].values
```

```
def minkowski_distance(A,B,p):

    total = 0

    for i in range(len(A)):
        total += abs(A[i]-B[i])**p

    return total**(1/p)
```

```
p_values = []
distances = []
```

```
for p in range(1,11):

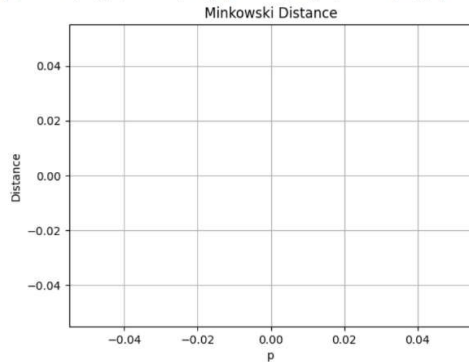
    d = minkowski_distance(A, B, p)
(A, B, p) A,B,p)

    p_values.append(p)
    distances.append(d)
```

```
print(distances)
```

```
plt.plot(p_values, distances,
marker='o')
plt.xlabel("p")
plt.ylabel("Distance")
plt.title("Minkowski Distance")
plt.grid()
plt.show()
```

```
np.float64(nan), np.float64(nan), np.float64(nan), np.float64(nan), np.float64(nan),
```



```
#A7
```

```
import pandas as pd
import numpy as np
import io
```

```
df =
pd.read_excel(io.BytesIO(uploaded[ '
Lab Session Data.xlsx' ]))
```

```
numeric_df =
df.select_dtypes(include='number')
```

```
A = numeric_df.iloc[0].values
B = numeric_df.iloc[1].values
```

```
def dot_product(A,B):
```

```
    total = 0
```

```
    for i in range(len(A)):
        total += A[i]*B[i]
```

```
    return total
```

```
print("Own Function =",
dot_product(A,B))
print("NumPy =", np.dot(A,B))
```

Output: Own Function = nan NumPy = nan

```
import pandas as pd
import numpy as np
import math
```

```
import io
```

```
df =
pd.read_excel(io.BytesIO(uploaded[ '
Lab Session Data.xlsx' ]))
```

```
numeric_df =
df.select_dtypes(include='number')
```

```
A = numeric_df.iloc[0].values
```

```
def euclidean_norm(A):
```

```
    total = 0
```

```
    for value in A:
        total += value**2
```

```
    return math.sqrt(total)
```

```
print("Own Function =",
euclidean_norm(A))
print("NumPy =", np.linalg.norm(A))
```

Output: Own Function = nan NumPy = nan

```
#A8
```

```
import pandas as pd
import math
import io
```

```
df =
pd.read_excel(io.BytesIO(uploaded[ '
Lab Session Data.xlsx' ]))
```

```
numeric_df =
df.select_dtypes(include='number')
```

```
data = numeric_df.iloc[:,0]
```

```
def mean(data):
```

```
    return sum(data)/len(data)
```

```
def variance(data):

    m = mean(data)

    return sum((x-m)**2 for x in
data)/len(data)

def standard_deviation(data):

    return
math.sqrt(variance(data))

print("Mean =", mean(data))
print("Variance =", variance(data))
print("Standard Deviation =",
standard_deviation(data))
Output: Mean = 19.8 Variance = 13.16
Standard Deviation = 3.627671429443411
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import io
df=pd.read_excel(io.BytesIO(uploade
d['Lab Session Data.xlsx']))
numeric_df =
df.select_dtypes(include='number')

feature = numeric_df.columns[0]

plt.hist(numeric_df[feature],
bins=10)

plt.xlabel(feature)
plt.ylabel("Frequency")
plt.title("Histogram")

plt.show()

print("Mean =",
np.mean(numeric_df[feature]))
print("Variance =",
np.var(numeric_df[feature]))
```

Output:

